IT CAREER HUB

Agile projects: Part 2

Учебный проект 2

- **Задача 1: Получение, обновление и удаление проекта** 2
- **Задача 2: Задачи для проекта, сотрудник с пагинацией, создание задачи** 5
- **Задача 3: Получение, обновление и удаление задачи** 12
- **Задача 4: Работа с файлами** 17
- **Задача 5: Получение файла, возможность его удаления** 25
- **Задача 6: Создание кастомного пользователя** 29

IT CAREER HUB

Учебный проект

Задача 1: Получение, обновление и удаление проекта

1. Создайте новый сериализатор **ProjectDetailSerializer** для получения конкретной информации по проекту:
    - **name**
    - **description**
    - **created_at**
    - **count_of_files**
2. Напишите новый класс-отображение **ProjectDetailAPIView** для получения конкретного проекта, обновления и удаления проекта.
3. При обновлении полученного проекта добавьте возможность частичного обновления полей.
4. Зарегистрируйте новый эндпоинт, проверьте как отрабатывают запросы **GET**, **PUT**, **DELETE.**
5. Зафиксируйте все изменения, сделайте запрос на слияние.

Решение:

1. Создайте новый сериализатор (apps/projects.serializers/project_serializers.py):

```python
class ProjectDetailSerializer(serializers.ModelSerializer):

    class Meta:
        model = Project
        fields = ('id', 'name', 'description', 'count_of_files')
```

2. Создайте класс-отображение ProjectDetailAPIView (apps/projects/views/project_views.py):

```python
class ProjectDetailAPIView(APIView):
    def get_object(self, pk: int):
        return get_object_or_404(Project, pk=pk)

    def get(self, request: Request, pk: int) -> Response:
        project = self.get_object(pk=pk)

        serializer = ProjectDetailSerializer(project)

        return Response(
            serializer.data,
            status=status.HTTP_200_OK,
        )

    def put(self, request: Request, pk: int) -> Response:
        project = self.get_object(pk=pk)

        serializer = CreateProjectSerializer(
            instance=project,
            data=request.data,
            partial=True
        )

        if serializer.is_valid(raise_exception=True):
            serializer.save()

            return Response(
                serializer.validated_data,
                status=status.HTTP_200_OK,
            )
```

```
        return Response(
            serializer.errors,
            status=status.HTTP_400_BAD_REQUEST,
        )

    def delete(self, request: Request, pk: int) -> Response:
        project = self.get_object(pk=pk)

        project.delete()

        return Response(
            data={
                "message": "Project was deleted successfully"
            },
            status=status.HTTP_200_OK,
        )
```

3. Зарегистрируйте новый эндпоинт (apps/project/urls.py):

```python
urlpatterns = [
   path('', ProjectsListAPIView.as_view()),
   path('<int:pk>/', ProjectDetailAPIView.as_view()), # NEW
]
```

4. Запустите сервер, проверьте **GET, PUT** и **DELETE** запросы.
5. Используйте Git команды (**git add, git commit, git push origin <branch_name>**).

IT CAREER HUB

Задача 2: Задачи для проекта, сотрудник с пагинацией, создание задачи

1. В приложении **tasks**, в модуле serializers создайте новый файл **tasks_serializers.py.**
2. Реализуйте сериализатор **AllTasksSerializer** для отображения краткой информации о задачах. Поля для отображения:
    - **name**
    - **status**
    - **priority**
    - **project** (название проекта)
    - **assignee** (email сотрудника)
    - **deadline**
3. Реализуйте сериализатор **CreateTaskSerializer** для создания новой задачи. Поля для создания новой задачи:
    - **name**
    - **description**
    - **priority**
    - **project(заполняется не по id, а по имени проекта)**
    - **tags**
    - **deadline**
4. Добавьте в сериализатор на создание новой задачи валидаторы полей:
    - **name** (минимальная длина)
    - **description** (минимальная длина)
    - **priority** (передаётся строго число, переданное число должно быть в доступных **choices**, от 1 до 5)
    - **project** (передаётся имя проекта, поиск среди всех проектов. Если такого имени нет - выводить сообщение об ошибке, что проекта не существует)
    - **tags** (список имён тегов, поиск среди всех тегов. Если тега по имени найдено не было - выводить сообщение об ошибке)
    - **deadline** (не может быть в прошлом)
5. Переопределите метод **create** для добавления информации тегов в промежуточную таблицу.
6. Напишите класс-отображение **AllTasksListAPIView** для получения списка всех задач, привязанных к конкретному проекту, или сотруднику на выбор. Реализуйте пагинацию для отображения только первых пяти записей на одной странице. Добавьте метод post для создания задачи.
7. Зарегистрируйте новый эндпоинт, проверьте как отрабатывают запросы **GET**, **POST.**
8. Зафиксируйте все изменения, сделайте запрос на слияние.

Решение:

1. Создание сериализатора (apps/tasks/serializers/task_serializers.py):

```python
from typing import Any

from rest_framework import serializers
from django.utils import timezone

from apps.projects.models import Project
from apps.tasks.models import Task, Tag
from apps.tasks.choices.priority import Priority


class AllTasksSerializer(serializers.ModelSerializer):
   project = serializers.SlugRelatedField(
       read_only=True,
       slug_field='name'
   )
   assignee = serializers.SlugRelatedField(
       read_only=True,
       slug_field='email'
   )

   class Meta:
       model = Task
       fields = (
           'id',
           'name',
           'status',
           'priority',
           'project',
           'assignee',
           'deadline'
       )


class CreateTaskSerializer(serializers.ModelSerializer):
   project = serializers.SlugRelatedField(
       slug_field='name',
       queryset=Project.objects.all(),
   )

   class Meta:
       model = Task
```

```
        fields = (
            'name',
            'description',
            'priority',
            'project',
            'tags',
            'deadline'
        )

        def validate_name(self, value: str) -> str:
            if len(value) < 10:
                raise serializers.ValidationError(
                    "The name of the task couldn't be less than 10 characters"
                )
            return value

        def validate_description(self, value: str) -> str:
            if len(value) < 50:
                raise serializers.ValidationError(
                    "The description of the task couldn't be less than 50
characters"
                )

            return value

        def validate_priority(self, value: int) -> int:
            if value not in [val[0] for val in Priority.choices()]:
                raise serializers.ValidationError(
                    "The priority of the task couldn't be one of the available
options"
                )

            return value

        def validate_project(self, value: str) -> str:
            if not Project.objects.filter(name=value).exists():
                raise serializers.ValidationError(
                    "The project with this name couldn't be found in the
database"
                )

            return value

        def validate_tags(self, value: list[str, ...]) -> list[str, ...]:
```

```
        if not Tag.objects.filter(name__in=value).exists():
            raise serializers.ValidationError(
                "The tags couldn't be found in the database"
            )

        return value

    def validate_deadline(self, value: str) -> int:
        value = timezone.make_aware(value, timezone.get_current_timezone())

        if value < timezone.now():
            raise serializers.ValidationError(
                "The deadline of the task couldn't be in the past"
            )

        return value

    def create(self, validated_data: dict[str, Any]) -> Task:
        tags = validated_data.pop('tags', [])

        task = Task.objects.create(**validated_data)

        for tag in tags:
            task.tags.add(tag)

        task.save()

        return task
```

2. Создание класса-отображения (apps/tasks/views/task_views.py):

```python
from django.db.models import QuerySet
from rest_framework.response import Response
from rest_framework.request import Request
from rest_framework import status
from rest_framework.views import APIView
from rest_framework.pagination import PageNumberPagination

from apps.tasks.models import Task
from apps.tasks.serializers.task_serializers import (
   AllTasksSerializer,
   CreateTaskSerializer
)


class StandardResultsSetPagination(PageNumberPagination):
   page_size = 5
   page_size_query_param = 'page_size'
   max_page_size = 15


class TasksListAPIView(APIView):
   paginator = StandardResultsSetPagination

   def get_objects(self) -> QuerySet:
       project_name = self.request.query_params.get('project_name')
       assignee_email = self.request.query_params.get('assignee_email')

       if project_name:
           return Task.objects.filter(project__name=project_name)

       elif assignee_email:
           return Task.objects.filter(assignee__email=assignee_email)

       else:
           return Task.objects.all()

   def get(self, request: Request, *args, **kwargs) -> Response:
       tasks = self.get_objects()

       if not tasks.exists():
           return Response(
               data=[],
               status=status.HTTP_204_NO_CONTENT
```

```
            )

        paginator = self.paginator()
        page = paginator.paginate_queryset(tasks, request, view=self)

        if page is not None:
            serializer = AllTasksSerializer(page, many=True)
            return paginator.get_paginated_response(serializer.data)

        serializer = AllTasksSerializer(tasks, many=True)

        return Response(
            serializer.data,
            status=status.HTTP_200_OK
        )

    def post(self, request: Request, *args, **kwargs) -> Response:
        serializer = CreateTasSerializer(data=request.data)

        if serializer.is_valid(raise_exception=True):
            serializer.save()

            return Response(
                serializer.data,
                status=status.HTTP_201_CREATED
            )

        return Response(
            serializer.errors,
            status=status.HTTP_400_BAD_REQUEST
        )
```

3. Регистрация класса-отображения (apps/tasks/urls.py):

```python
from apps.tasks.views.task_views import TasksListAPIView


urlpatterns = [
   path('', TasksListAPIView.as_view()),  # NEW
   path('tags/', TagListAPIView.as_view()),
   path('tags/<int:pk>/', TagDetailAPIView.as_view()),
]
```

4. Запустите сервер, проверьте **GET, POST** запросы.
5. Используйте Git команды (**git add, git commit, git push origin <branch_name>**).

IT CAREER HUB

Задача 3: Получение, обновление и удаление задачи

1. Напишите новый сериализатор **TaskDetailSerializer**:
    - Сериализатор должен принимать все поля, кроме **updated_at**, **deleted_at**
    - Более подробное отображение информации о проекте(вложенный сериализатор **ProjectShortInfoSerializer**)
2. В уже написанном сериализаторе **CreateTaskSerializer** поменяйте название на **CreateUpdateTaskSerializer**, добавьте метод update на обновление существующей задачи.
3. Создайте классовое отображение **TaskDetailAPIView** для получения полной информации о конкретной задаче по её **ID**, возможности полного, или частичного обновления задачи и удаления задачи.
4. Зарегистрируйте новый эндпоинт, проверьте как отрабатывают запросы **GET**, **PUT, DELETE.**
5. Зафиксируйте все изменения, сделайте запрос на слияние.

Решение:

1. Создание сериализатора TaskDetailSerializer(apps/tasks/serializers/task_serializers.py):

```python
class TaskDetailSerializer(serializers.ModelSerializer):
   project = ProjectShortInfoSerializer()
   tags = TagSerializer(many=True, read_only=True)

   class Meta:
       model = Task
       exclude = ('updated_at', 'deleted_at')
```

2. Создание сериализатора ProjectShortInfoSerializer(apps/projects/serializers/project_serializers.py):

```python
class ProjectShortInfoSerializer(serializers.ModelSerializer):
   class Meta:
       model = Project
       fields = ('id', 'name')
```

3. Обновление сериализатора CreateTaskSerializer(apps/tasks/serializers/task_serializers.py):

```python
class CreateUpdateTaskSerializer(serializers.ModelSerializer):
   project = serializers.SlugRelatedField(
       slug_field='name',
       queryset=Project.objects.all(),
   )

   …

   def update(self, instance: Task, validated_data: dict[str, Any]) -> Task:
      tags = validated_data.pop('tags', [])

      for attr, value in validated_data.items():
          setattr(instance, attr, value)

      if tags:
          for tag in tags:
              instance.tags.add(tag)

      instance.save()

      return instance
```

4. Создание классового отображения TaskDetailAPIView(apps/tasks/views/task_views.py):

```python
class TaskDetailAPIView(APIView):
    def get_object(self):
        return get_object_or_404(Task, pk=self.kwargs['pk'])

    def get(self, request: Request, *args, **kwargs) -> Response:
        task = self.get_object()

        serializer = TaskDetailSerializer(task)

        return Response(
            serializer.data,
            status=status.HTTP_200_OK
        )

    def put(self, request: Request, *args, **kwargs) -> Response:
        task = self.get_object()

        serializer = CreateUpdateTaskSerializer(
            instance=task,
            data=request.data,
            partial=True
        )

        if serializer.is_valid(raise_exception=True):
            serializer.save()

            return Response(
                serializer.data,
                status=status.HTTP_200_OK
            )

        return Response(
            serializer.errors,
            status=status.HTTP_400_BAD_REQUEST
        )

    def delete(self, request: Request, *args, **kwargs) -> Response:
        task = self.get_object()

        task.delete()

        return Response(
```

```
            data={
                "message": "The task has been deleted."
            },
            status=status.HTTP_204_NO_CONTENT
        )
```

5. Запустите сервер, проверьте **GET, PUT** и **DELETE** запросы.
6. Используйте Git команды (**git add, git commit, git push origin <branch_name>**).

IT CAREER HUB

Задача 4: Работа с файлами

1. В приложении **projects** создайте новый модуль **utils.**
2. Создайте в этом модуле файл **upload_file_helpers.py.**
3. В этом файле необходимо создать функции:
    - Функция на проверку валидных расширений файла
    - Функция на проверку размерности файла (файл не должен быть больше **2 MB**)
    - Функция на создание пути для сохранения файла
    - Функция на сохранение файла (файл должен записываться по частям для избежания потенциальных ошибок и потери данных)
4. В модуле **serializers** в приложении **projects** создайте новый файл **project_file_serializers.py.**
5. Создайте сериализатор **AllProjectFilesSerializer** для отображения краткой информации по всем файлам:
    - **id**
    - **file_name**
    - **project**
6. Создайте сериализатор **CreateProjectFileSerializer** для создания нового файла:
    - Для создания необходимо только поле **file_name**
    - Поле **file_name** необходимо валидировать:
        - Проверка на наличие символов из таблицы **ASCII**
        - Проверка на доступные расширения файлов (**pdf, csv, doc, xlsx**)
7. Переопределите метод **create**:
    - Перед сохранением файла необходимо создать путь для сохранения
    - Так же необходимо получить из тела запроса сам файл и проверить его на размерность (**не более 2 MB**)
    - Если файл проходит все проверки - сохранить его в проекте, сохранить объект файла
    - если файл слишком большой - поднять исключение валидации с сообщением об ошибке
8. Напишите класс-отображение **ProjectFileListGenegicView** для получения списка всех файлов:
    - Реализовать метод на получение всех файлов для конкретного проекта по имени проекта. Если имя проекта не было передано - выводить список полностью всех файлов
9. Добавьте в класс-отображение **ProjectFileListGenegicView** метод **post** для создания нового файла и привязки его к конкретному проекту (проект получать через поле **id**, которое будет передаваться в теле запроса).
10. Зарегистрируйте новый эндпоинт, проверьте как отрабатывают запросы **GET**, **POST.**
11. Зафиксируйте все изменения, сделайте запрос на слияние.

Решение:

1. Создайте файл upload_file_helpers.py с необходимыми функциями (apps/projects/utils/upload_file_helpers.py):

```python
import os
from pathlib import Path


ALLOWED_EXTENSIONS = ['.csv', '.doc', '.pdf', '.xlsx', '.py']


def check_extension(filename):
    extension = Path(filename).suffix

    if extension not in ALLOWED_EXTENSIONS:
        return False

    return True


def check_file_size(file, required_size=2):
    file_size = file.size / (1024 * 1024)

    if file_size > required_size:
        return False

    return True


def create_file_path(project_name, file_name):
    new_file_name, file_ext = file_name.split('.')

    file_path = "documents/{}/{}.{}".format(
        project_name.replace(' ', '_')
        new_file_name.replace(' ', '_'),
        file_ext
    )

    return file_path


def save_file(file_path, file_content):
    os.makedirs(os.path.dirname(Path(file_path)), exist_ok=True)
```

```
    with open(file_path, 'wb') as f:
        for chunk in file_content.chunks():
            f.write(chunk)

    return file_path
```

2. Создайте новые сериализаторы (apps/projects/serializers/project_file_serializers.py):

```python
from rest_framework import serializers

from apps.projects.models import ProjectFile
from apps.projects.utils.upload_file_helpers import (
   check_extension,
   create_file_path,
   check_file_size,
   save_file
)


class AllProjectFilesSerializer(serializers.ModelSerializer):

   class Meta:
       model = ProjectFile
       fields = ('id', 'file_name', 'project')


class CreateProjectFileSerializer(serializers.ModelSerializer):
   class Meta:
       model = ProjectFile
       fields = ('file_name',)

   def validate_file_name(self, value: str) -> str:
       if not value.isascii():
           raise serializers.ValidationError(
               "Please, provide a valid file name."
           )
       if not check_extension(value):
           raise serializers.ValidationError(
               "Valid file extensions: ['.csv', '.doc', '.pdf', '.xlsx']"
           )

       return value

        def create(self, validated_data):
       project = self.context.get('project')
       raw_file = self.context.get('raw_file')

       file_path = create_file_path(
           project_name=project.name,
           file_name=validated_data['file_name']
```

```
        )

        if check_file_size(file=raw_file):
            save_file(file_path=file_path, file_content=raw_file)

            validated_data['file_path'] = file_path

            project_file = ProjectFile.objects.create(**validated_data)

            project_file.project.add(project)

            return project_file

        else:
            raise serializers.ValidationError(
                "File size is too large (2 MB as maximum)."
            )
```

3. Создайте новый класс-отображение (apps/projects/views/project_file_views.py):

Python

```python
from rest_framework.request import Request
from rest_framework.response import Response
from rest_framework import status
from rest_framework.generics import get_object_or_404, ListCreateAPIView

from apps.projects.models import ProjectFile, Project
from apps.projects.serializers.project_file_serializers import (
   AllProjectFilesSerializer,
   CreateProjectFileSerializer,
)


class ProjectFileListGenericView(ListCreateAPIView):
   def get_serializer_class(self, *args, **kwargs):
       if self.request.method == 'GET':
           serializer = AllProjectFilesSerializer

           return serializer

       elif self.request.method == 'POST':
           serializer = CreateProjectFileSerializer

           return serializer

   def get_queryset(self):
       project_name = self.request.query_params.get('project')

       if project_name:
           project_file = ProjectFile.objects.filter(
               project__name=project_name
           )
           return project_file

       return ProjectFile.objects.all()

   def list(self, request: Request, *args, **kwargs) -> Response:
       project_files = self.get_queryset()
       if not project_files.exists():
           return Response(
               data=[],
               status=status.HTTP_204_NO_CONTENT
           )
```

```
        serializer = self.get_serializer(project_files, many=True)

        return Response(
            serializer.data,
            status=status.HTTP_200_OK
        )

    def create(self, request: Request, *args, **kwargs) -> Response:
        file_content = request.FILES["file"]
        project_id = request.data["project_id"]
        request.data['file_name'] = file_content.name if file_content else None

        project = get_object_or_404(Project, pk=project_id)

        serializer = self.get_serializer(
            data=request.data,
            context={
                "raw_file": file_content,
                "project": project
            }
        )

        if serializer.is_valid(raise_exception=True):
            serializer.save()

            return Response(
                data={
                    "message": "File upload successfully"
                },
                status=status.HTTP_200_OK
            )

        else:
            return Response(
                data=serializer.errors,
                status=status.HTTP_400_BAD_REQUEST
            )
```

4. Зарегистрируйте новый эндпоинт (apps/projects/urls.py):

```python
from apps.projects.views.project_file_views import (
   ProjectFileListAPIView,
)

urlpatterns = [
   path('', ProjectsListAPIView.as_view()),
   path('<int:pk>/', ProjectDetailAPIView.as_view()),
   path('files/', ProjectFileListAPIView.as_view()),
]
```

5. Запустите сервер, проверьте **GET, POST** запросы.
6. Используйте Git команды (**git add, git commit, git push origin <branch_name>**).

IT CAREER HUB

Задача 5: Получение файла, возможность его удаления

1. Создайте сериализатор **ProjectFileDetailSerializer** для получения подробной информации о файле:
    - **name**
    - **created_at**
    - **project (вложенный сериализатор для отображения названия проекта)**
2. Напишите классовое отображение **ProjectFileDetailGenericView** для получения информации о конкретном файле и проекте, к которому он привязан, а также возможности удаления этого файла, как из базы данных, так и из проекта.
3. Зарегистрируйте новый эндпоинт, проверьте как отрабатывают запросы **GET**, **DELETE.**
4. Зафиксируйте все изменения, сделайте запрос на слияние.

Решение:

1. Создание сериализатора (apps/projects/serializers/project_file_serializer.py):

```python
class ProjectFileDetailSerializer(serializers.ModelSerializer):
   project = ProjectShortInfoSerializer(many=True)


   class Meta:
       model = ProjectFile
       exclude = ('file_path',)
```

2. Создание отображения ProjectFileDetailGenericView (apps/projects/views/project_file_views.py):

Python

```python
class ProjectFileDetailGenericView(RetrieveDestroyAPIView):
   serializer_class = ProjectFileDetailSerializer

   def get_object(self):
       return get_object_or_404(ProjectFile, pk=self.kwargs['pk'])

   def retrieve(self, request: Request, *args, **kwargs) -> Response:
       task = self.get_object()

       serializer = self.get_serializer(task)

       return Response(
           data=serializer.data,
           status=status.HTTP_200_OK
       )

   def destroy(self, request: Request, *args, **kwargs) -> Response:
       task = self.get_object()

       try:
           delete_file(file_path=task.file_path.path)

       except Exception as e:
           return Response(
               data={"message": str(e)},
               status=status.HTTP_400_BAD_REQUEST
           )
       else:
           task.delete()

       return Response(
           data={
               "message": "File deleted successfully"
           },
           status=status.HTTP_200_OK
       )
```

3. Добавление функции на удаление файла (apps/projects/utils/upload_file_helpers.py):

```python
def delete_file(file_path):
   os.remove(os.path.realpath(file_path))
```

4. Регистрация нового эндпоинта(apps/projects/urls.py)

```python
from apps.projects.views.project_file_views import (
   ProjectFileListGenericView,
   ProjectFileDetailGenericView,
)


urlpatterns = [
   path('', ProjectListAPIView.as_view()),
   path('<int:pk>/', ProjectDetailAPIView.as_view()),
   path('files/', ProjectFileListGenericView.as_view()),
   path('files/<int:pk>/', ProjectFileDetailGenericView.as_view()),  # NEW
]
```

5. Запуск сервера, проверка **GET, DELETE** запросов.
6. Git команды (**git add, git commit, git push origin <branch_name>**).

IT CAREER HUB

Задача 6: Создание кастомного пользователя

1. Удалите существующую базу.
2. Создайте новое приложение **users**, зарегистрируйте его в проекте.
3. Создайте свою собственную модель **User**, в которой будут поля:
    - **username (**Строковое поле, не более 50 символов, уникальное, обязательно к заполнению**)**
    - **first_name (**строковое поле, не более 40 символов, обязательно к заполнению**)**
    - **last_name (**строковое поле, не более 40 символов, обязательно к заполнению**)**
    - **email (**поле email, уникальное, обязательное к заполнению)
    - **phone** (строковое поле, не более 75 символов, не обязательно к заполнению**)**
    - **is_staff (**административное логическое поле, по умолчанию False**)**
    - **is_active (**логическое поле, по умолчанию True**)**
    - **date_joined (**Поле даты и времени, заполняется автоматически при создании**)**
    - **last_login (**Поле даты и времени, заполняется при входе в систему**)**
    - **updated_at (**Поле даты и времени, заполняется автоматически при всех обновлениях**)**
    - **deleted_at (**Поле даты и времени, заполняется только если поле deleted переходит в состояние True**)**
    - **deleted (**Логическое поле, по умолчанию - True**)**
    - **project (**Связь с моделью **Project**. При удалении проекта все сотрудники так же удаляются, связующее поле у проекта **- users,** при создании пользователя проекта у него может и не быть**)**
    - **position (**строковое поле выбора из готовых вариантов позиций:
        - **Ceo**
        - **Cto**
        - **Designer**
        - **Programmer**
        - **Product_owner**
        - **Project_owner**
        - **Project_manager**
        - **Qa)**
4. Укажите поле email, как поле для входа в систему.
5. Добавьте дополнительные поля для регистрации:
    - username
    - last_name
    - first_name
6. Перепишите настройки django на работу с пользователем в файле settings.py.

7. Переопределите связь с новым пользователем в других моделях, где была связь с пользователем (модель **Task**).
8. Проведите миграции и убедитесь, что всё работает.

Решение:

1. Создание нового проекта:

```python
python manage.py startapp users
```

2. Создание своего пользователя (apps/users/models.py):

```python
from django.contrib.auth.base_user import AbstractBaseUser
from django.contrib.auth.models import PermissionsMixin, UserManager
from django.core.validators import MinLengthValidator
from django.db import models
from django.utils.translation import gettext_lazy as _

from apps.projects.models import Project
from apps.users.choices.positions import UserPositions


class User(AbstractBaseUser, PermissionsMixin):
   username = models.CharField(
       _("username"),
       max_length=50,
       unique=True,
       error_messages={
           "unique": _("A user with that username already exists."),
       }
   )
   first_name = models.CharField(
       _("first name"),
       max_length=40,
       validators=[MinLengthValidator(2)],
   )
   last_name = models.CharField(
       _("last name"),
       max_length=40,
       validators=[MinLengthValidator(2)],
   )
   email = models.EmailField(
       _("email address"),
       max_length=150,
       unique=True
   )
```

```
    phone = models.CharField(max_length=75, null=True, blank=True)
    is_staff = models.BooleanField(default=False)
    is_active = models.BooleanField(default=True)
    date_joined = models.DateTimeField(
        name="registered", auto_now_add=True
    )
    last_login = models.DateTimeField(null=True, blank=True)
    updated_at = models.DateTimeField(auto_now=True)
    deleted_at = models.DateTimeField(null=True, blank=True)
    deleted = models.BooleanField(default=False)
    position = models.CharField(
        max_length=15,
        choices=UserPositions.choices,
        default=UserPositions.PROGRAMMER
    )
    project = models.ForeignKey(
        Project,
        on_delete=models.CASCADE,
        related_name="users",
        null=True,
        blank=True,
)


    USERNAME_FIELD = "email"
    REQUIRED_FIELDS = [
        "username",
        "first_name",
        "last_name",
        “position”
     ]


    objects = UserManager()

    def __str__(self):
        return f"{self.last_name} {self.first_name}"
```

3. Создание и применение миграций:

```python
python manage.py makemigrations
python manage.py migrate
```

4. Проверка, что всё хорошо, создание суперпользователя:

```python
python manage.py createsuperuser
```